Realising, Visualising and Applying Quantum Gates - Derivations

Will Noble

June 2026

Contents

1 An Important Message

These notes have been provided as a supplementary resource for my poster - Realising, Visualising and Applying Quantum Gates. This is purposefully calculation heavy, with possibly trivial knowledge being explained in full for the sake of completeness, such as the inclusion of trigonometric formulas that you see during A Levels.

Text highlighted in Red has been highlighted to present important results, most likely being seen within the poster, such as the table of realised single qubit gates. When realising single qubit gates, parameters will always be ordered γ, τ, ϵ.

There is also supplementary Python code, written with the following packages: NumPy (Harris et al. 2020), Matplotib, QuTip (Lambert et al. 2026), CUDA-Q (NVIDIA 2026), SciPy (Virtanen et al. 2020), ImageIO (ImageIO Contributors 2026), OS and Qiskit (Javadi-Abhari et al. 2024). This has been used to numerically confirm results, and has been featured in some chapters in order to justify reasoning.

FOR ACADEMIC HONESTY: THE PROCESS TO PRODUCE GIFS IN PYTHON WERE ASSISTED BY CHATGPT DUE TO A LACK OF UNDERSTANDING WITH THE PACKAGES USED. ADDITIONALLY, inner_product, global_phase_comparison AND qft_matrix REQUIRED HELP FROM CHATGPT TO DEFINE THEM CORRECTLY.

2 Prerequisites

We must initially define the Pauli matrices (Barnett 2026b, p. 15):

$$\sigma_X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \sigma_Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \qquad \sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \tag{1}$$

It is important to understand what a Quantum state $|\phi\rangle$ or $\langle\phi|$ is. We define $|\phi\rangle$ as a column vector representation, and $\langle\phi|$ as a row vector representation (Barnett 2026a, p. 6). This becomes important to know when we consider the solutions to the Time-dependent Schrödinger equation, as it only makes sense to multiply vectors / matrices in the order (2x2) · (2x1).

For our 1-qubit gates, we should define:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \qquad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

This is the orthonormal basis for $\mathbb{R}^2$ and classically, we would define $|0\rangle$ and $|1\rangle$ as results of a coin flip (Barnett 2026a, p. 4), with them representing heads and tails respectively. It is quite useful to know that qubits are very similar to classical bits, yet they can be states other than 0 and 1, such as $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$.

Some additional rules to these states, which we explain in the 2-qubit case in order to get the idea (Barnett 2026a, p. 6):

$$|\phi\rangle = \alpha_1|00\rangle + \alpha_2|01\rangle + \alpha_3|10\rangle + \alpha_4|11\rangle, \quad \sum_i |\alpha_i|^2 = 1$$

Where $|00\rangle, |01\rangle, |10\rangle, |11\rangle$ is the orthonormal basis for a 2-qubit system (Barnett 2026a, p. 12).

It might be useful to know that $|0\rangle \otimes |0\rangle = |00\rangle$, where $\otimes$ is the tensor product and the result $|00\rangle = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}$.

Generally we can define the tensor product $\otimes$ as (Barnett 2026b, p. 4):

$$\begin{pmatrix} a \\ b \end{pmatrix} \otimes \begin{pmatrix} c \\ d \end{pmatrix} = \begin{pmatrix} a \begin{pmatrix} c \\ d \end{pmatrix} \\ b \begin{pmatrix} c \\ d \end{pmatrix} \end{pmatrix} \tag{2}$$

The use of the tensor product will become useful when deriving a new definition for the Quantum Fourier Transform.

From our definitions above, the size of an n-qubit state is $N = 2^n$. This links to the dimension of Hilbert Spaces, which will not be discussed here.

We also define the Exact Hamiltonian (Barnett 2026b, p. 14). This has a physical representation corresponding to a driven two-level system, where:

ϵ: Energy level spacing

ω: Driving frequency

γ: Coupling strength

$$\mathcal{H}_{exact}(t) = \frac{\epsilon}{2}\sigma_Z + \gamma\sigma_X cos(\omega t) \tag{3}$$

Defined from 1 and considering a physical system.

It must be noted that $\hbar = 1$ for everything in this derivation and the poster.

Finally, we want to define the Time-dependent Schrödinger equation (Barnett 2026b, p. 9):

$$i\hbar\frac{\partial}{\partial t}|\psi(t)\rangle = \mathcal{H}(t)|\psi(t)\rangle \tag{4}$$

We can obtain the following solution from the Time-dependent Schrödinger equation:

$$|\psi(t)\rangle = e^{-i\mathcal{H}}|\psi(0)\rangle \tag{5}$$

We must recognise that this doesn't necessarily work for a Time-dependent Hamiltonian, yet we will not be using this apart from comparing two quantum states later on. This can easily be solved by hand using matrix exponentiation, but I also solved it in Python, which we will use later.

For a Time-dependent Hamiltonian, the following definition makes more sense (Cappellaro 2012, p. 4):

$$|\psi(t)\rangle = \mathcal{T}exp(-i\int_0^t \mathcal{H}(t')dt')|\psi(0)\rangle \tag{6}$$

Where $\mathcal{T}$ is the Time Ordering operator. This is derived via a Dyson Series, yet we will not look at explaining this in too much detail. Simply, it orders quantum operators that a multiplied together in chronological order, which is important as these quantum operators may not commute with each other. The

time-dependent series was solved with Python, as I was not able to obtain the result myself. This is dropped for RWA as the RWA is time-independent.

3 Obtaining the Rotation Adjusted Hamiltonian

We start with 3, 4. We want to use (Barnett 2026b, p. 18)(Documentation 2025):

$$|\psi'(t)\rangle = e^{\frac{i\sigma_Z \omega t}{2}} |\psi(t)\rangle \tag{7}$$

which is the rotating frame for ω.

Before we go further, it is important to understand the concept of a rotating frame. "This means that we consider a frame of resonance in which all state vectors are rotating themselves with the resonance frequency of the system." - (Documentation 2025). This is like changing the coordinate system you're working with by applying a unitary transformation.

We say that:

$$\mathcal{R}(t) = e^{\frac{i\sigma_Z \omega t}{2}} \tag{8}$$

from 7.

Apply the product rule for derivatives and obtain:

$$\frac{\partial}{\partial t}|\psi'(t)\rangle = \frac{\partial \mathcal{R}}{\partial t}|\psi(t)\rangle + \mathcal{R}\frac{\partial}{\partial t}|\psi(t)\rangle \tag{9}$$

Multiply by i to obtain:

$$i\frac{\partial}{\partial t}|\psi'(t)\rangle = i\frac{\partial \mathcal{R}}{\partial t}|\psi(t)\rangle + i\mathcal{R}\frac{\partial}{\partial t}|\psi(t)\rangle \tag{10}$$

Using 7 and 4 we obtain:

$$i\frac{\partial}{\partial t}|\psi'(t)\rangle = i\frac{\partial \mathcal{R}}{\partial t}\mathcal{R}^{-1}|\psi'(t)\rangle + \mathcal{R}\mathcal{H}_{exact}|\psi(t)\rangle$$

$$i\frac{\partial}{\partial t}|\psi'(t)\rangle = (i\frac{\partial \mathcal{R}}{\partial t}\mathcal{R}^{-1} + \mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1})|\psi'(t)\rangle \tag{11}$$

from substituting the rotating frame adjustment twice.

We can now say:

$$\mathcal{H}_{rot} = i\frac{\partial \mathcal{R}}{\partial t}\mathcal{R}^{-1} + \mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1} \tag{12}$$

4 Obtaining the Rotating Wave Approximation Hamiltonian

$$\mathcal{R}^{-1} = e^{\frac{-i\omega\sigma_Z t}{2}} \tag{13}$$

$$\frac{\partial R}{\partial t} = \frac{i\omega\sigma_Z}{2}\mathcal{R} \tag{14}$$

Substituting 13 and 14 into 12:

$$\mathcal{H}_{rot} = \frac{-\omega}{2}\sigma_Z + \mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1} \tag{15}$$

Recalling 3, we can substitute this into $\mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1}$:

$$\mathcal{R}\mathcal{H}_{exact}\mathcal{R}^{-1} = \frac{\epsilon}{2}\mathcal{R}\sigma_Z\mathcal{R}^{-1} + \gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1} \tag{16}$$

What is $\mathcal{R}$ (8) as a matrix?:

$$\mathcal{R}(t) = e^{\frac{i\sigma_Z\omega}{2}} = I + \frac{i\sigma_z\omega}{1!}t + \frac{(i\sigma_Z\omega)^2}{2!}t^2 + ... \tag{17}$$

using the Taylor series for e^x.

Recall (via 1):

$$\sigma_Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$$

$$\sigma_Z{}^2 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} = I \tag{18}$$

Via 17:

$$\mathcal{R}(t) = (I - \frac{\sigma_Z{}^2}{2!}(\frac{\omega t}{2})^2 + ...) + i(\frac{\sigma_Z}{1!}\frac{\omega t}{2} - \frac{\sigma_Z{}^3}{3!}(\frac{\omega t}{2})^3 + ...) \tag{19}$$

Recalling the Taylor series for $\sin(x)$ and $\cos(x)$:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - ... \quad , \quad \cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - .... \tag{20}$$

Using 19 and 20:

$$\mathcal{R}(t) = I\cos(\frac{\omega t}{2}) + i\sigma_Z \sin(\frac{\omega t}{2})$$

$$\mathcal{R}^{-1}(t) = I\cos(\frac{\omega t}{2}) - i\sigma_Z \sin(\frac{\omega t}{2})$$

$$\sigma_Z\mathcal{R}^{-1} = \sigma_Z\cos(\frac{\omega t}{2}) - iI\sin(\frac{\omega t}{2})$$

$$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = (I\cos(\frac{\omega t}{2}) + i\sigma_Z \sin(\frac{\omega t}{2}))(\sigma_Z\cos(\frac{\omega t}{2}) - iI\sin(\frac{\omega t}{2}))$$

$$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = \sigma_Z\cos^2(\frac{\omega t}{2}) - iI^2\cos(\frac{\omega t}{2})\sin(\frac{\omega t}{2}) + i\sigma_Z^2\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2}) + \sigma_Z\sin^2(\frac{\omega t}{2})$$

$$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = \sigma_z(\cos^2(\frac{\omega t}{2}) + \sin^2(\frac{\omega t}{2}))$$

$$\mathcal{R}\sigma_Z\mathcal{R}^{-1} = \sigma_Z \tag{21}$$

using $\sin^2(\theta) + \cos^2(\theta) = 1$

There is probably an easier way to do this, but it's the first way I found, ignoring any potential commutativity of the matrices.

Now we have (via 15, 16 and 21):

$$\mathcal{H}_{rot}(t) = \frac{\epsilon - \omega}{2}\sigma_z + \gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1} \tag{22}$$

Now we want to find $\mathcal{R}\sigma_X\mathcal{R}^{-1}$:

$$\sigma_X\mathcal{R}^{-1} = \sigma_X\cos(\frac{\omega t}{2}) - i\sigma_X\sigma_Z\sin(\frac{\omega t}{2})$$

$$\mathcal{R}\sigma_X\mathcal{R}^{-1} = (I\cos(\frac{\omega t}{2}) + i\sigma_Z \sin(\frac{\omega t}{2}))(\sigma_X\cos(\frac{\omega t}{2}) - i\sigma_X\sigma_Z\sin(\frac{\omega t}{2}))$$

$$\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X\cos^2(\frac{\omega t}{2}) - i\sigma_X\sigma_Z\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2}) + i\sigma_Z\sigma_X\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2}) + \sigma_Z\sigma_X\sigma_Z\sin^2(\frac{\omega t}{2}) \tag{23}$$

We must calculate the matrices $-i\sigma_X\sigma_Z$, $i\sigma_Z\sigma_X$ and $\sigma_Z\sigma_X\sigma_Z$ using 1:

$$-i\sigma_X\sigma_Z = -i\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix} = -i\begin{pmatrix}0 & -1\\ 1 & 0\end{pmatrix} = -\sigma_Y$$

$$i\sigma_Z\sigma_X = i\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix} = i\begin{pmatrix}0 & 1\\ -1 & 0\end{pmatrix} = -\sigma_Y$$
$$\sigma_Z\sigma_X\sigma_Z = \begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix}\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix}\begin{pmatrix}1 & 0\\ 0 & -1\end{pmatrix} = -\begin{pmatrix}0 & 1\\ 1 & 0\end{pmatrix} = -\sigma_X \tag{24}$$

Now we can find $\mathcal{R}\sigma_X\mathcal{R}^{-1}$ (23):

$$\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X\cos^2(\frac{\omega t}{2}) - \sigma_X\sin^2(\frac{\omega t}{2}) - 2\sigma_Y\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2})$$
$$\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X(\cos^2(\frac{\omega t}{2}) - \sin^2(\frac{\omega t}{2})) - \sigma_Y(2\sin(\frac{\omega t}{2})\cos(\frac{\omega t}{2})) \tag{25}$$

Double Angle Formulae:

$$\cos(2\theta) = \cos^2(\theta) - \sin^2(\theta), \qquad \sin(2\theta) = 2\sin(\theta)\cos(\theta)$$
$$\mathcal{R}\sigma_X\mathcal{R}^{-1} = \sigma_X\cos(\omega t) - \sigma_Y\sin(\omega t) \tag{26}$$

Now (using 22 and 26):

$$\gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1} = \gamma\sigma_X\cos^2(\omega t) - \gamma\sigma_Y\cos(\omega t)\sin(\omega t)$$
$$\cos(2\theta) = 2\cos^2(\theta) - 1, \qquad \cos^2(\theta) = \frac{\cos(2\theta)+1}{2}$$
$$\gamma\cos(\omega t)\mathcal{R}\sigma_X\mathcal{R}^{-1} = \frac{\gamma}{2}\sigma_X + \frac{\gamma}{2}\sigma_X\cos(2\omega t) - \frac{\gamma}{2}\sigma_Y\sin(2\omega t) \tag{27}$$

Using 22 (Barnett 2026b, p. 18):

$$\mathcal{H}_{rot}(t) = \frac{1}{2}(\epsilon-\omega)\sigma_Z + \frac{\gamma}{2}\sigma_X + V(t) \tag{28}$$

Where (from 28):

$$V(t) = \frac{\gamma}{2}(\sigma_X\cos(2\omega t) - \sigma_Y\sin(2\omega t)) \tag{29}$$

When we drop the V(t) from $\mathcal{H}_{rot}$, you obtain $\mathcal{H}_{RWA}$.

Thus we obtain the time independent Hamiltonian (Barnett 2026b, p. 18):

$$\mathcal{H}_{RWA} = \frac{1}{2}(\epsilon-\omega)\sigma_Z + \frac{\gamma}{2}\sigma_X \tag{30}$$

5 Why can we drop the time-dependent V(t)?

We suddenly dropped the time dependent part of $\mathcal{H}_{rot}$ (28), the $V(t)$ (29). Why do we want to do this?

The motivation for this is to have a time independent Hamiltonian, such that time leaves it unaffected when we try and tune parameters for logic gates.

In order to see whether this is a viable change, we must compare the solutions of the Time-dependent Schrödinger equation. These have been calculated in Python using the SciPy (Virtanen et al. 2020) package.

```
def inner_product(f, g, frame_transform_f=False, frame_transform_g=False):
    """
    This function will compute the inner product of 2 functions, which helps us measure the overlap of the functions.
    It is often seen by the notation <f,g>. For our examples, the inner product will be defined on [0,T]. The T will be our time interval in which we solved the ODE earlier.
    If you want to use an exact Hamiltonian, you will want to set frame_transform to the function (set to True) in order for everything to be in a rotating frame.
    """

    fidelity = []
    for i in range(f.shape[1]):
        ψ_f = f[:, i] # This rotates the exact function into the rotating frame
        ψ_g = g[:, i]
        if frame_transform_f is True:
            rot_adj_f = rotating_frame_adjustment(t_axis[i])
            ψ_f = rot_adj_f @ ψ_f
            # This ensures we get the ψ' that we need
        if frame_transform_g is True:
            rot_adj_g = rotating_frame_adjustment(t_axis[i])
            ψ_g = rot_adj_g @ ψ_g
        overlap = np.vdot(ψ_f, ψ_g) # For vdot, check https://numpy.org/doc/stable/reference/generated/numpy.vdot.html
        fidelity.append(np.abs(overlap)**2)
    fidelity = np.array(fidelity)
    average_fidelity = trapezoid(fidelity, t_axis) / (t_axis[-1] - t_axis[0]) # This computes an average
    fidelityerror = 1 - min(fidelity)
    return average_fidelity, fidelityerror
```

Figure 1: Inner Product Function

We use the idea of Fidelity: "Fidelity in quantum computing measures the accuracy of quantum operations – essentially how closely real quantum processes match the ideal, error-free processes."(Ivezic 2023)

We find that for the following tuples (in the order $(\epsilon, \gamma, \omega)$), we obtain the following Fidelity and Max Fidelity Error:
(11, 1, 10), 0.9980862569699225, 0.006619300129882544
(41, 1, 40), 0.9998696553789628, 0.000448888096092781
(1001, 1, 1000), 0.9999995285788046, 1.244694879054542e-06

The following tuples were used to respect an inequality mentioned in Section 5, and these are seen in Lecture 2 (Barnett 2026b, p. 17).

A Fidelity of 0.99 or greater is seen as optimal, and it can be seen that this is far exceeded by the Rotating Wave Approximation Hamiltonian (30) when compared to the Exact Hamiltonian (3) in the solution to the Schrödinger equation (5 and 6).

However, there's a slight hitch in our justification. We've only chosen a small T, $T = 10$ in the following calculations:

$$\bar{F} = \frac{1}{T}\int_0^T F(t)dt, \qquad F(t) = |\langle\psi_{rot}(t)|\psi_{RWA}(t)\rangle|^2, \qquad 0 \leq \bar{F} \leq 1, T \in \mathbb{R}_{\geq 0} \tag{31}$$

It's really important to understand where $F(t)$ comes from. This can be understood by considering Born Rule (Barnett 2026a, p. 9):

$$p_n = |\langle\phi_n|\psi\rangle|^2 = \langle\phi_n|\rho|\phi_n\rangle \tag{32}$$

Where p_n is the probability of measuring state $|\phi_n\rangle$ and $\rho = |\psi\rangle\langle\psi|$.

From this definition, we can see that F(t) (31) is the probability of measuring the state $|\psi_{rot}\rangle$ in the system $|\psi_{RWA}\rangle$. A high $F(t)$ implies that the quantum states are close to each other. It would then be natural to ask why we use $\bar{F}$ (31). This is because we want to compute a time-average, not just Fidelity at a single point, otherwise we aren't really extracting much information about the whole state as time evolves.

You might also ask, why is $\bar{F}$ an integral? This is due to the consideration of the sum of values of the function on finite points. As we increase the number of points to infinity, this results in the integral we see above (31). In our case, we want the average of all $F(t)$ across the closed interval $[0, T]$. In Python, we actually do perform a sum in the form of the Trapezium Rule (you will have seen this in A-Level Maths or equivalent), as the numerical integral is much more computationally difficult. Additionally, I chose 10,000 points on $[0, T]$ in order to compute $\bar{F}$, which balances accuracy and computation time.

We've used a small T, and thus our justification isn't actually all that strong. If we pick $T = 1000$, our justification actually breaks down for a small tuple (11,1,10). However, for (1001,1,1000), we obtain 0.998675399136747, 0.003971685201408226, our respective Fidelity and Maximum Fidelity error results, so we get the idea that as we increase T, we should increase our ϵ and ω.

It seems like a good idea to find values of F (Fidelity) and graph it against values of T, in order to see whether our assumption holds for large T, but this is computationally very time consuming. Instead, we consider the following relationship (the time-average of $V(t)$):

$$\int_0^T V(t)dt \approx 0 \tag{33}$$

So we want to calculate:

$$\int_0^T \frac{\gamma}{2}(\sigma_X \cos(2\omega t) - \sigma_Y \sin(2\omega t))dt$$

$$\frac{\gamma}{2}(\sigma_X \frac{\sin(2\omega t)}{2\omega} + \sigma_Y \frac{\cos(2\omega t)}{2\omega}) \tag{34}$$

with $t = 0$, $t = T$.

This results in:

$$\frac{\gamma}{2}(\sigma_X \frac{\sin(2\omega T)}{2\omega} + \sigma_Y \frac{\cos(2\omega T)}{2\omega} - \sigma_Y \frac{1}{2\omega})$$

If we take $\lim_{\omega\to\infty}$ with $\lim_{T\to\infty}$ and we know that $|\sin(x)|\leq 1$ and $|\cos(x)|\leq 1$ $\forall x$, then we find that $\int_0^T V(t)dt$ (33) will tend to 0.

What we can deduce for this is for a large T, in order to achieve the Rotating Wave Approximation (30) and drop $V(t)$ (29), we also want to achieve a sufficiently large ω.

We also want (Barnett 2026b, p. 17): $\sqrt{(\epsilon-\omega)^2+\gamma^2} \ll \omega$, so ϵ and γ must also be picked accordingly.

Finally, this isn't actually the end of the story for $V(t)$. There are other edge cases which lead to the RWA failing, but these will not stop our realisation of quantum gates, for now.

6 Deriving the Hadamard and Phase Gate

Realised gates have the form (Barnett 2026b, p. 20):

$$U(\tau) = e^{-iH_{RWA}\tau} \tag{35}$$

A unitary operator, the propagator, can be used to derive this result (Cappellaro 2020).

We must have the following inequality hold as well (mentioned earlier):

$$\sqrt{(\epsilon-\omega)^2+\gamma^2} \ll \omega \tag{36}$$

For convenience:

$$\Delta = \sqrt{(\epsilon-\omega)^2+\gamma^2}. \tag{37}$$

We compare these to our Sought gates (Barnett 2026b, p. 16):

$$H = \frac{1}{\sqrt{2}}(\sigma_X+\sigma_Z), \qquad P = e^{-i\phi\sigma_Z} \tag{38}$$

We start with P, the Phase Gate (Williams 2011, p. 77).

Comparing U (35) and P (38):

$$\phi\sigma_Z = \mathcal{H}_{RWA}\tau \tag{39}$$

Recalling 30, we must pick $\gamma = 0$ and $\epsilon \neq \omega$, with $\epsilon \ll \omega$.

Comparing:

$$\frac{1}{2}(\epsilon-\omega)\tau = \phi \implies \tau = \frac{2\phi}{\epsilon-\omega} \tag{40}$$

So the Phase Gate is realised for:

$$\gamma = 0, \qquad \tau = \frac{2\phi}{\epsilon-\omega}, \qquad \epsilon \neq \omega, \qquad \epsilon \ll \omega \tag{41}$$

We want to find the Hadamard gate now (Williams 2011, p. 94).

Ideally, we want to represent 35 as a function of matrices, not as an matrix exponential (using 30):

$$U(\tau) = I - i\frac{\mathcal{H}_{RWA}\tau}{1!} - \frac{(\mathcal{H}_{RWA}\tau)^2}{2!} + i\frac{(\mathcal{H}_{RWA}\tau)^3}{3!} + ...$$

$$U(\tau) = (I - \frac{(\mathcal{H}_{RWA}\tau)^2}{2!} + ...) - i(\frac{\mathcal{H}_{RWA}\tau}{1!} - \frac{(\mathcal{H}_{RWA}\tau)^3}{3!} + ...) \tag{42}$$

It makes sense to ask what ${\mathcal{H}_{RWA}}^2$ is:

$${\mathcal{H}_{RWA}}^2 = (\frac{1}{2}(\epsilon - \omega)\sigma_Z + \frac{\gamma}{2}\sigma_X)(\frac{1}{2}(\epsilon - \omega)\sigma_Z + \frac{\gamma}{2}\sigma_X)$$

$${\mathcal{H}_{RWA}}^2 = \frac{1}{4}(\epsilon - \omega)^2{\sigma_Z}^2 + \frac{\gamma}{4}\sigma_Z\sigma_X + \frac{\gamma}{4}\sigma_X\sigma_Z + \frac{\gamma^2}{4}{\sigma_X}^2 \tag{43}$$

We only need to check ${\sigma_X}^2$ (using 1):

$${\sigma_X}^2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = I$$

$${\mathcal{H}_{RWA}}^2 = \frac{1}{4}(\epsilon - \omega)^2 I + \frac{i\gamma}{4}\sigma_Y - \frac{i\gamma}{4}\sigma_Y + \frac{\gamma^2}{4}I$$

$${\mathcal{H}_{RWA}}^2 = \frac{1}{4}((\epsilon - \omega)^2 + \gamma^2)I = \frac{\Delta^2}{4}I$$

$$U(\tau) = (I - \frac{1}{2!}(\frac{\tau\Delta}{2})^2 I + ...) - i(\frac{\mathcal{H}_{RWA}\tau}{1!} - \frac{(\mathcal{H}_{RWA}\tau)^3}{3!} + ...)$$

$$U(\tau) = (I - \frac{1}{2!}(\frac{\tau\Delta}{2})^2 I + ...) - \frac{2i\mathcal{H}_{RWA}}{\Delta}(\frac{1}{1!}(\frac{\tau\Delta}{2}) - \frac{1}{3!}(\frac{\tau\Delta}{2})^3 + ...)$$

$$U(\tau) = I\cos(\frac{\tau\Delta}{2}) - \frac{2i\mathcal{H}_{RWA}}{\Delta}\sin(\frac{\tau\Delta}{2}) \tag{44}$$

using Taylor series of $\cos(x)$ and $\sin(x)$, from 20.

We now want to substitute in Δ (37) and $\mathcal{H}_{RWA}$ (30):

$$U(\tau) = I\cos(\frac{\tau\sqrt{(\epsilon - \omega)^2 + \gamma^2}}{2}) - i(\frac{(\epsilon - \omega)\sigma_Z + \gamma\sigma_X}{(\sqrt{(\epsilon - \omega)^2 + \gamma^2})})\sin(\frac{\tau\sqrt{(\epsilon - \omega)^2 + \gamma^2}}{2}) \tag{45}$$

We have now obtained an easier form of $U(\tau)$ to work with.

Comparing H (38) and U (35):

$$\frac{\tau\Delta}{2} = \frac{(2k+1)\pi}{2}, \qquad k \in \mathbb{Z} \tag{46}$$

in order for $\cos(\frac{\tau\Delta}{2}) = 0$.

We find that:

$$\tau = \frac{(2k+1)\pi}{\Delta}, \qquad \gamma = \epsilon - \omega \tag{47}$$

This results in:

$$\gamma = \epsilon - \omega, \qquad \tau = \frac{(2k+1)\pi}{\sqrt{2}\gamma}, \qquad \epsilon \neq \omega, \qquad \epsilon \ll \omega \tag{48}$$

After substituting in Δ (37).

It must be noted that we are left with an iH or $-iH$, however this is a global phase difference that does not matter as the Quantum gate is functionally the same. Scalar differences of $e^{i\phi}$ are not significant on the realisation of the gate. This is mentioned by Nielsen and Chuang. (Nielsen and Chuang 2010, p. 93)

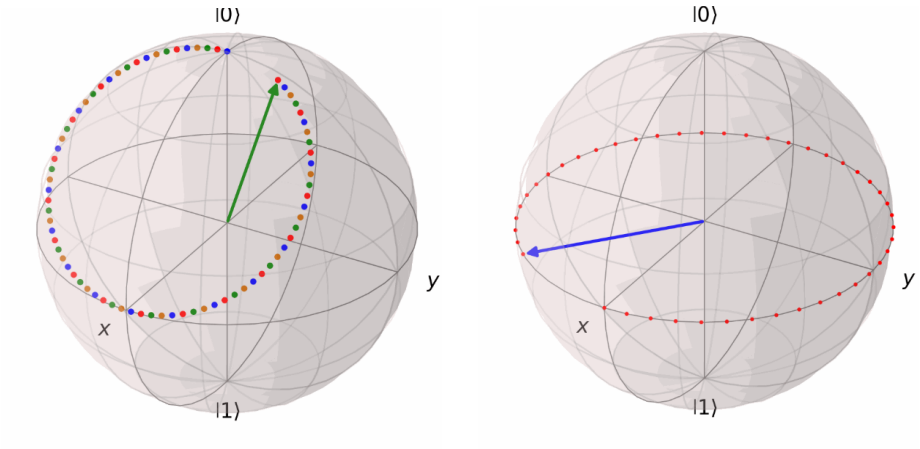
```python
def global_phase_comparison(U, V, tol=1e-8):
    """
    This function takes in 2 matrices that you want to compare the properties of, disregards the global phase difference.
    """
    overlap = np.vdot(U.flatten(), V.flatten()) # Flatten changes the matrices to a 4x1 column vector
    if np.abs(overlap) < tol:
        return False # This checks whether the flattened matrices are orthogonal.
    phase = overlap / np.abs(overlap)
    return np.allclose(U, V / phase, atol=tol)
```

Figure 2: Global Phase Comparison

```python
if np.isclose(ε, ω):
    print("Cannot realise the Hadamard Gate for ε = ω.")
else:
    if (global_phase_comparison(hadamard, realised_func(np.pi/(np.sqrt(2) * (ε - ω))))):
        print("The Hadamard Gate has been realised for τ = π/(sqrt(2) * γ), γ = ε - ω.")
        print(f"The gate fidelity is {gate_fidelity(hadamard, realised_func(np.pi/(np.sqrt(2) * (ε - ω))))}.")
    else:
        print("The Hadamard Gate hasn't been realised due to a choice of τ and γ.")
        print(f"The gate fidelity is {gate_fidelity(hadamard, realised_func(np.pi/(np.sqrt(2) * (ε - ω))))}.")
```

Figure 3: Hadamard Phase Comparison, trivial example for $k = 1$

7 The Bloch Sphere



(a) Hadamard Gate (b) Phase Gate

Figure 4: Bloch sphere representations of the gates using QuTip (Lambert et al. 2026), NumPy (Harris et al. 2020), ImageIO (ImageIO Contributors 2026) and OS Python packages.

Now we want to briefly look at a common representation of Quantum gates, in the form of the Bloch Sphere.

Recalling the Pauli matrices, σ_X, σ_Y and σ_Z, it can be determined that these result in π radians anticlockwise rotations in their respective axis after exponentiation if you consider a continuous time evolution. These do result in π radians anticlockwise rotations anyway, but not seen on the Bloch Sphere without exponentiation first.

Considering the Hadamard gate, applying H twice, as seen in Figure 4(a), must return the identity as the initial state $|0\rangle$ is the start and end of the trajectory. After 1 Hadamard gate, the state the qubit is in is $|+\rangle$.

We can confirm our expectation by evaluating H^2 (38 and 24):

$$H^2 = \frac{1}{2}(\sigma_X + \sigma_Z)(\sigma_X + \sigma_Z)$$

$$H^2 = \frac{1}{2}({\sigma_X}^2 + \sigma_X\sigma_Z + \sigma_Z\sigma_X + {\sigma_X}^2)$$

$$H^2 = \frac{1}{2}(I + i\sigma_Y - i\sigma_Y + I)$$

$$H^2 = I \tag{49}$$

In the case of the Phase Gate, this is entirely a rotation about the Z-axis. If you were to start with $|0\rangle$ or $|1\rangle$, they would be unaffected by the Phase gate.

It is really important to recognise that the Hadamard gate is represented by a matrix, and does not display this time evolution in practice. This time evolution has been provided as a way of understanding how the gate acts on the Bloch Sphere, obtained via exponentiation.

8 Realising other 1-Qubit Gates

ALL GATES REALISED BELOW HAVE BEEN CHECKED IN PYTHON WITH THEIR DEFINITION. PYTHON FILE SUPPLEMENTED ELSEWHERE.

We've now obtained the Hadamard and Phase gates. From these, we can obtain other gates from these, or from the pre-defined Pauli Matrices. Using the substitution $\phi = \frac{\pi}{2}$ (38), you can obtain the Phase S gate, and $\phi = \frac{\pi}{4}$ (38), obtaining the $\frac{\pi}{8}$ gate. It should also be noted that $\phi = \pi$ (38) will obtain the σ_Z gate, which we've already defined (1).

Before we go further, it is useful to realise that the Phase gate has a linear transformation representation as:

$$P(\phi) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\phi} \end{pmatrix} \tag{50}$$

This is fundamentally the same to an alternative definition, found by representing the exponential sought form of the Phase gate as a matrix, and noticing that these definitions differ by a global phase which is irrelevant physically.

We now have 7 1-qubit gates, the X/NOT Gate (Williams 2011, p. 72), Y, Z (1), S, Phase, Hadamard (38) and $\frac{\pi}{8}/T$ gates.

We can also realise the NOT gate from our earlier $U(\tau)$ (45):

$$U(\tau) = Icos(\frac{\tau\sqrt{(\epsilon-\omega)^2+\gamma^2}}{2}) - i(\frac{(\epsilon-\omega)\sigma_Z + \gamma\sigma_X}{(\sqrt{(\epsilon-\omega)^2+\gamma^2})})sin(\frac{\tau\sqrt{(\epsilon-\omega)^2+\gamma^2}}{2})$$

Using:

$$\gamma \neq 0, \qquad \tau = \frac{(2k+1)\pi}{\gamma}, \qquad \epsilon = \omega, \qquad k \in \mathbb{Z} \tag{51}$$

Obtains the X/NOT gate.

Trivially:

$$\gamma = 0, \qquad \epsilon = \omega \tag{52}$$

Results in the identity gate I.

Now let's consider the $\sqrt{X}$ gate (Williams 2011, p. 72). We will find out what $\sqrt{\sigma_X}$ is, which we will achieve by diagonalisation:

$$\sigma_X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad \chi_{\sigma_X}(x) = x^2 - 1 \tag{53}$$

Such that its eigenvalues are:

$$\lambda_1 = 1, \qquad \lambda_2 = -1 \tag{54}$$

With some calculations:

$$\vec{v_1} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \vec{v_2} = \begin{pmatrix} 1 \\ -1 \end{pmatrix} \tag{55}$$

Using the Diagonalisation formula seen in Linear Algebra and Groups - Term 2 (David Evans 2023, p. 26), we can obtain $\sigma_X = PDP^{-1}$, where:

$$D = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \qquad P = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad P^{-1} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix} \tag{56}$$

We now write:

$$\sqrt{\sigma_X} = PD^{\frac{1}{2}}P^{-1} \tag{57}$$

So

$$\sqrt{\sigma_X} = \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \begin{pmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{pmatrix} \tag{58}$$

It is interesting to note here that $\sqrt{\sigma_X} = HSH$ (38 and substitution of $\phi = \frac{\pi}{2}$), seen by square rooting $\frac{1}{2}$, using these coefficients on P and P^{-1}.

Thus:

$$\sqrt{\sigma_X} = \frac{1}{2} \begin{pmatrix} 1+i & 1-i \\ 1-i & 1+i \end{pmatrix} \tag{59}$$

After multiplying matrices.

From 35:

$$U(\tau) = e^{-iH_{RWA}\tau}$$

Substitute our values for the X gate (51), such that:

$$\sigma_X = e^{\frac{-i(2k+1)\pi\sigma_X}{2}} \tag{60}$$

So we can square root both sides and obtain:

$$\sqrt{\sigma_X} = e^{\frac{-i(2k+1)\pi\sigma_X}{4}} \tag{61}$$

We therefore pick:

$$\gamma \neq 0, \qquad \tau = \frac{(2k+1)\pi}{2\gamma}, \qquad \epsilon = \omega, \qquad k \in \mathbb{Z} \tag{62}$$

achieving $\sqrt{X}$ gate.

We can now defined our 3 Rotation gates (Williams 2011, p. 77):

$$R_X(\theta) = e^{\frac{-i\sigma_X\theta}{2}}, \qquad R_Y(\theta) = e^{\frac{-i\sigma_Y\theta}{2}}, \qquad R_Z(\theta) = e^{\frac{-i\sigma_Z\theta}{2}} \tag{63}$$

We start with deriving $R_X(\theta)$ and $R_Z(\theta)$ (using 35 and 63):

$$\frac{\sigma_X \theta}{2} = (\frac{1}{2}(\epsilon - \omega)\sigma_Z + \frac{\gamma}{2}\sigma_X)\tau \tag{64}$$

It is clear to see that $\epsilon = \omega$

Now $\frac{\theta}{2} = \frac{\gamma\tau}{2}$, such that $\tau = \frac{\theta}{\gamma}$.

Overall, $R_X(\theta)$ is realised for:

$$\gamma \neq 0, \qquad \tau = \frac{\theta}{\gamma}, \qquad \epsilon = \omega \tag{65}$$

The $R_Z(\theta)$ gate is fundamentally identical to the Phase gate (38), with a realisation that is very similar.

Now we define the $R_Y(\theta)$ gate as:

$$R_Y(\theta) = S R_X(\theta) S^\dagger \tag{66}$$

In the poster, we define $R_Y(\theta)$ with the same parameters as $R_X(\theta)$. We do this assuming that we can take S optimally, such that using the identity only involves the tuning parameters of $R_X(\theta)$. Otherwise we have to consider both parameters of S (38, 41 and substitution of $\phi = \frac{\pi}{2}$) and $R_X(\theta)$ (65).

I will also define $U(\theta, \phi, \lambda)$ as (Williams 2011, p. 82):

$$U(\theta, \phi, \lambda) = R_Z(\phi) R_Y(\theta) R_Z(\lambda) \tag{67}$$

However the derivation for this will not be included in the document.

The two gates defined above are identities rather than definitions, but they will not be covered due to limitations of the $\mathcal{H}_{exact}$ (3).

Finally, to bridge between quantum gates and Quantum Fourier Transforms (QFTs), if we have the parameters:

$$\gamma = 0, \qquad \tau = \frac{4\pi}{2^k(\epsilon - \omega)}, \qquad \epsilon \neq \omega, \qquad \epsilon \ll \omega, \qquad k \in \mathbb{N} \tag{68}$$

We obtain the Dyadic Rational Phase Gate (Nielsen and Chuang 2010, p. 218), which is very important when considering the circuit implementation of the QFT.

This has the form:

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{\frac{2\pi i}{2^k}} \end{pmatrix} \tag{69}$$

We've used the linear transformation definition of the Phase gate (50) to find its parameters.

9 The Quantum Fourier Transform

Finally we can introduce the Quantum Fourier Transform. First, we must define what a Discrete Fourier Transform is (Nielsen and Chuang 2010, p. 217):

$$y_k \equiv \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{\frac{2\pi i j k}{N}} x_j \tag{70}$$

Where we define the following terms:
y_k: Transformed complex vector of data
N: Length of initial vector
x_k: Initial complex vector of data
k: The element of y_k you wish to read, with $0 \leq k \leq N$

Similarly, we define the Quantum Fourier Transform (QFT) as (Nielsen and Chuang 2010, p. 217):

$$|j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{\frac{2\pi i j k}{N}} |k\rangle \tag{71}$$

Where we define the following terms:
$|j\rangle$: Complex state of input data
N: Length of initial vector
$|k\rangle$: Basis state that appears in transformed data

We want to define $N = 2^n$ where $n \in \mathbb{Z}$, preferably $n \geq 1$, where n is the number of qubits in a quantum register.

"The action on an arbitrary state may be written" as (Nielsen and Chuang 2010, p. 217):

$$\sum_{j=0}^{N-1} x_j |j\rangle \mapsto \sum_{k=0}^{N-1} y_k |k\rangle \tag{72}$$

If we use the following notation conventions (Nielsen and Chuang 2010, p. 218):
$|j\rangle$, a state with binary representation $j = j_1 j_2 ... j_n$
$j = j_1 2^{n-1} + j_2 2^{n-2} + ... + j_n 2^0$ as a more formal representation.
$0.j_l j_{l+1} ... j_m$ representing the binary fraction $\frac{j_l}{2} + \frac{j_{l+1}}{4} + ... + \frac{j_m}{2^{m-l+1}}$

We use these binary representations, as it directly links to R_k, where each application of a rotation angle is of the form $2\pi(\frac{1}{2} + \frac{1}{4} + ... + \frac{1}{2^n})$ if R_2 to R_n are applied.

We want to derive a new expression for the QFT using tensor products (2). This has been taken exactly from (Nielsen and Chuang 2010, p. 218):

$$|j\rangle \mapsto \frac{1}{2^{\frac{n}{2}}} \sum_{k=0}^{2^n-1} e^{\frac{2\pi i j k}{2^n}} |k\rangle \tag{73}$$

Using our substitution of $N = 2^n$.

$$|j\rangle = \frac{1}{2^{\frac{n}{2}}} \sum_{k_1=0}^{1} ... \sum_{k_n=0}^{1} e^{2\pi i j(\sum_{l=1}^{n} k_l 2^{l-1})} |k_1 ... k_n\rangle \tag{74}$$

Using the tensor product:

$$|j\rangle = \frac{1}{2^{\frac{n}{2}}} \sum_{k_1=0}^{1} ... \sum_{k_n=0}^{1} \bigotimes_{l=1}^{n} e^{2\pi i j k_l 2^{-l}} |k_l\rangle \tag{75}$$

Using billinearity:

$$|j\rangle = \frac{1}{2^{\frac{n}{2}}} \bigotimes_{l=1}^{n} [\sum_{k_l=0}^{1} e^{2\pi i j k_l 2^{-l}}] |k_l\rangle \tag{76}$$

$$|j\rangle = \bigotimes_{l=1}^{n} [|0\rangle + e^{2\pi i j 2^{-l}} |1\rangle] \tag{77}$$

Thus our new definition of the QFT:

$$|j\rangle \mapsto \frac{(|0\rangle + e^{2\pi i 0.j_n})(|0\rangle + e^{2\pi i 0.j_{n-1} j_n} |1\rangle) ... (|0\rangle + e^{2\pi i 0.j_1 j_2 ... j_n})}{2^{\frac{n}{2}}} \tag{78}$$

Note: This derivation has been included for the sake of completeness, considering its importance on the poster. This will not be explained further on this document.

Now we want to understand how to obtain a quantum circuit out of this definition by working backwards (Nielsen and Chuang 2010, p. 219).

We have our input state $|j_1 ... j_n\rangle$, and we apply a Hadamard (38) to the first qubit, producing:

$$\frac{1}{2^{\frac{1}{2}}} (|0\rangle + e^{2\pi i 0.j_1} |1\rangle) |j_2 ... j_n\rangle \tag{79}$$

It is quite useful to know that $e^{2\pi i 0.j_1} = -1$ for $j_1 = 1$ and $+1$ otherwise (Nielsen and Chuang 2010, p. 219). This is intuitive when considering what H (38) does to $|0\rangle$ and $|1\rangle$.

Applying an R_2 gate (69) produces:

$$\frac{1}{2^{\frac{1}{2}}} (|0\rangle + e^{2\pi i 0.j_1 j_2} |1\rangle) |j_2 ... j_n\rangle \tag{80}$$

Now we apply the R_3 gate through to R_n, resulting in:

$$\frac{1}{2^{\frac{1}{2}}} (|0\rangle + e^{2\pi i 0.j_1 j_2 ... j_n} |1\rangle) |j_2 ... j_n\rangle \tag{81}$$

Now applying a Hadamard to the second qubit:

$$\frac{1}{2^{\frac{2}{2}}}(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle)(|0\rangle + e^{2\pi i 0.j_2}|1\rangle)|j_3 \ldots j_n\rangle \tag{82}$$

Now we apply the R_2 through R_{n-1} gates on the second qubit:

$$\frac{1}{2^{\frac{2}{2}}}(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle)(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle)|j_3 \ldots j_n\rangle \tag{83}$$

If we repeat this process on every subsequent qubit, we get the final state:

$$\frac{1}{2^{\frac{n}{2}}}(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle)(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle)\ldots(|0\rangle + e^{2\pi i 0.j_n}|1\rangle) \tag{84}$$

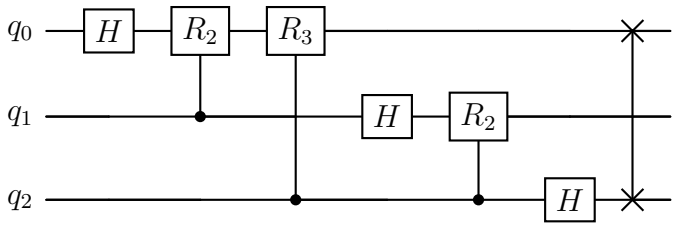
Now, it may not be obvious, but the SWAP gate (Williams 2011, p. 57) needs to be applied to reverse the order of the qubits, ordering them correctly:

$$\frac{1}{2^{\frac{n}{2}}}(|0\rangle + e^{2\pi i 0.j_n}|1\rangle)(|0\rangle + e^{2\pi i 0.j_{n-1} j_n}|1\rangle)\ldots(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle) \tag{85}$$

What about determining the time complexity of the QFT?

We apply a Hadamard, then $n-1$ R_k gates on qubit 1, so n gates. Then, we apply a Hadamard, then $n-2$ R_k gates on qubit 2, so $n-1$ gates. We see that there are $n+(n-1)+(n-2)+\ldots 1$ gates, thus we obtain $\frac{n(n+1)}{2}$ gates before SWAP gates are applied. Applying the SWAP gates, we apply $\lfloor \frac{n}{2} \rfloor$, which is negligible when considering time complexity (the reason for the floor symbol is due to an odd number of qubits not swapping the middle qubit). Thus we have $\Theta(n^2)$ time complexity for the QFT.

Below, I've provided a visual for a 3-qubit QFT in order to visualise what's actually going on in a quantum circuit:



This plot was produced using Qiskit (Javadi-Abhari et al. 2024) and verified using CUDA-Q (NVIDIA 2026) Python packages. This was done by creating a PNG using Qiskit and creating a LaTeX form in CUDA-Q, as well as obtaining a quantum state that we can compare against the matrix that would achieve the same QFT.

From this, we can see exactly what's happening to q_0, q_1 and q_2, and it agrees with the theory. It is also interesting to know that $R_2 = S$ gate and $R_3 = T$ gate.

It would also be quite useful to present the matrix that achieves this result (Nielsen and Chuang 2010, p. 220):

$$QFT_8 = \frac{1}{\sqrt{8}} \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & \omega & \omega^2 & \omega^3 & \omega^4 & \omega^5 & \omega^6 & \omega^7 \\ 1 & \omega^2 & \omega^4 & \omega^6 & 1 & \omega^2 & \omega^4 & \omega^6 \\ 1 & \omega^3 & \omega^6 & \omega & \omega^4 & \omega^7 & \omega^2 & \omega^5 \\ 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 & 1 & \omega^4 \\ 1 & \omega^5 & \omega^2 & \omega^7 & \omega^4 & \omega & \omega^6 & \omega^3 \\ 1 & \omega^6 & \omega^4 & \omega^2 & 1 & \omega^6 & \omega^4 & \omega^2 \\ 1 & \omega^7 & \omega^6 & \omega^5 & \omega^4 & \omega^3 & \omega^2 & \omega \end{pmatrix} \tag{86}$$

Where a QFT matrix, QFT_N uses $N = 2^n$, so a QFT matrix on a 3-qubit register uses QFT_8 as its notation. This was tested in Python on $|000\rangle$, or e_1 in $\mathbb{R}^8$.

If we also consider the Fast Fourier Transform (FFT), the classical way of achieving a Discrete Fourier Transform, it has the time complexity $\Theta(N \log_2 N)$. This means the FFT "requires exponentially more operations to compute the Fourier transform on a classical computer than it does to implement the quantum Fourier transform on a quantum computer" (Nielsen and Chuang 2010, p. 220).

However, run into a bit of a problem. There is no known way to implement the QFT because "the amplitudes in a quantum computer cannot be directly accessed by measurement. Thus, there is no way of determining the Fourier transformed amplitudes of the original state. Worse still, there is in general no way to efficiently prepare the original state to be Fourier transformed." (Nielsen and Chuang 2010, p. 220)

A simple explanation for this is that if you apply a QFT on an input state $|j\rangle$, your output, when measured, will only obtain 1 state. When you repeat the experiment, you might obtain another state when you re-measure. In order to get the distribution of the fully transformed data, you need to execute many measurements, which defeats the purpose of the exponential advantage of the QFT.

10 Extensions

In my poster, I mentioned some extensions such as Shor's algorithm, Grover's algorithm and limitations of the QFT. Below, I will give a small insight into some other downsides of the QFT, and in general, Quantum computers.

The QFT, as mentioned in The Quantum Fourier Transform, is not actually as efficient as it seems, considering the probabilistic nature of quantum states. However, there are more issues to do with Quantum computers.

Qubits are extremely sensitive, and require very specific conditions to function normally. Factors such as noise, inaccurate measurements or inaccurate gates can hinder the efficiency of a quantum computer. Notably, some quantum computers use superconducting qubits in order to operate, which are outrageously expensive (considering you need near 0K temperatures) and only a few companies have access to them, such as IBM or Google. There are other methods for a functioning quantum computer, but I won't mention them right now.

Directly linking to the realisation of quantum gates, small fidelity errors (consider gate_fidelity function in my Python code) on large quantum circuits can result in much larger errors that hinder the usage of a quantum computer. If you consider a 30-qubit quantum computer, you already have 480 quantum gates needed for a QFT. Consider a much larger n-qubit quantum computer and the errors can become noticeably large.

It becomes very difficult to obtain very high values for the parameters ϵ and ω (and a suitable γ) when considering H_{exact} (3), which can be the main reason for gate fidelity being too high.

THESE NOTES ARE OFF OF ADDITIONAL KNOWLEDGE OBTAINED THROUGHOUT THIS PROJECT. THERE AREN'T PROPER REFERENCES FOR THIS SECTION YET.

11 Citations

References

Barnett, Ryan (2026a). *Probabilities and Quantum Mechanics: Lecture 1*. Lecture notes, Department of Mathematics, Imperial College London.

— (2026b). *Probabilities and Quantum Mechanics: Lecture 2*. Lecture notes, Department of Mathematics, Imperial College London.

Cappellaro, Paola (2012). *22.51 Quantum Theory of Radiation Interactions: Chapter 5 - Time Evolution*. MIT OpenCourseWare, accessed 9 June 2026. URL: https://ocw.mit.edu/courses/22-51-quantum-theory-of-radiation-interactions-fall-2012/485cf9e8ede00033f1c7d7b6eb6fecb2_MIT22_51F12_Ch5.pdf.

— (2020). *6.1: Time Dependent Schrodinger Equation*. Date not entirely known, last updated in 2020. Accessed: June 14th 2026. URL: https://phys.libretexts.org/Bookshelves/Nuclear_and_Particle_Physics/Introduction_to_Applied_Nuclear_Physics_(Cappellaro)/06%3A_Time_Evolution_in_Quantum_Mechanics/6.01%3A_Time-dependent_Schrodinger_equation.

David Evans, Michele Zordan (2023). *MATH40003, Linear Algebra and Groups Term 2, 2022-23, Part I **. Lecture notes, Department of Mathematics, Imperial College London.

Documentation, qopt (2025). *Rabi Driving*. Accessed: June 14th 2026. URL: https://qopt.readthedocs.io/en/latest/examples/rabi_rwa.html.

Harris, Charles R. et al. (Sept. 2020). "Array programming with NumPy". In: *Nature* 585.7825, pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: https://doi.org/10.1038/s41586-020-2649-2.

ImageIO Contributors (2026). *ImageIO*. Version 2.37.3. Python package for reading and writing image data. URL: https://imageio.readthedocs.io.

Ivezic, Marin (2023). *Fidelity in Quantum Computing*. URL: https://postquantum.com/quantum-computing/fidelity-quantum/.

Javadi-Abhari, Ali et al. (2024). "Quantum Computing with Qiskit". In: *arXiv preprint arXiv:2405.08810*.

Lambert, Neill et al. (2026). "QuTiP 5: The Quantum Toolbox in Python". In: *Physics Reports* 1153, pp. 1–62. ISSN: 0370-1573. DOI: 10.1016/j.physrep.2025.10.001. URL: https://www.sciencedirect.com/science/article/pii/S0370157325002704.

Nielsen, Michael A. and Isaac L. Chuang (2010). *Quantum Computation and Quantum Information*. 10th Anniversary Edition. The Edinburgh Building, Cambridge CB2 8RU, UK: Cambridge University Press.

NVIDIA (2026). *CUDA-Q: A Hybrid Quantum-Classical Programming Platform*. Version 13.2. URL: `https://nvidia.github.io/cuda-quantum/`.

Virtanen, Pauli et al. (2020). "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python". In: *Nature Methods* 17, pp. 261–272. DOI: `10.1038/s41592-019-0686-2`.

Williams, Colin P. (2011). *Explorations in Quantum Computing*. 2nd Edition. London: Springer-Verlag London Ltd: Springer London.